

Indice

1. Introducción.....	2
1.1 Herramientas usadas.....	2
2. Dominio.....	2
2.1 Jobs.....	2-3
3. Implementación.....	3
3.1 Decisiones de diseño.....	3
3.2 El inbox, channel y dispatcher.....	3-4
4. cmd.....	4
4.1 cmd/server/main.go().....	4-5
5. Internal/Queue.....	5
5.1 Internal/queue/job.go().....	5
5.2 Internal/queue/status.go().....	5
5.3 Internal/queue/queue.go().....	5-7
5.4 Internal/queue/worker.go().....	7-8
5.5 Internal/queue/process.go().....	8-9
5.6 cmd/server/registry.go().....	9
5.7 cmd/server/dlq.go().....	9
6. Internal/api.....	10
6.1 Internal/api/server.go().....	10
6.2 Internal/api/jobs.go().....	10
6.3 Internal/api/response.go().....	10

1. Introducción

El presente documento reúne la documentación y explicación del proyecto.

Este proyecto fue realizado con fines de aprendizaje, con el objetivo de estudiar el manejo de servers y protocolos HTTP en el lenguaje de programación Golang. También, se buscaba profundizar en técnicas de diseño aprendidas como patrones de diseño y desarrollo vía TDD.

1.1 Herramientas usadas:

- Go 1.22 o superior (usa ``net/http.ServeMux`` con path variables, disponible desde 1.22)
- Sin dependencias externas. Todo el proyecto usa únicamente la librería estándar de Go.
- Claude Code para optimización de métodos.

2. Dominio

El programa es una cola de trabajos que recibe trabajos mediante protocolos HTTP (GET/POST) y los procesa de forma concurrente con una pool de workers (Seteamos 4 workers concurrentes por parámetro) y reintenta automáticamente los que fallan con backoff exponencial.

Si un trabajo agota sus reintentos (5 reintentos) cae a una Dead-Letter Queue consultable y reintentable manualmente.

2.1 Jobs

Hay dos tipos de trabajos implementados:

- EmailJob:** Es un envío de mail que en este caso será simulado.

-ImageResizeJob: Redimensión de imágenes real (jpeg y png) usando el algoritmo nearest neighbor. Prioriza simplicidad y testeabilidad sobre calidad visual. En escalados grandes (por ejemplo, una imagen 4K reducida a 200x200) el resultado se ve notoriamente pixelado. Un algoritmo bilinear daría mejor calidad.

3. Implementación

3.1 Decisiones de diseño

El flujo de trabajo fue hecho mediante Test Driven Development, siguiendo el patrón de escribir un primer test en rojo que falle y luego implementar el método, para refactorizarlo una vez que el test de verde.

Se usaron dos patrones de diseño: Strategy y Registry.

Esto con el fin de que los distintos trabajos estén desacoplados de toda la lógica de la cola, misma que solo sabe recibir trabajos independientemente de cuáles sean.

La idea principal era tener una estructura que permita agregar futuros trabajos si se así se quisiera.

3.2 El inbox, channel y dispatcher.

El programa puede dividirse en tres partes:

-El inbox (Queue):

Es el buzón de entrada de las distintas tareas, cuando llega un trabajo nuevo es el primer lugar al que entra. Es ilimitado y nunca se bloquea por más que haya una o N tareas esperando.

Las tareas son encoladas con Submit()

Por qué es crítico que sea ilimitado: quien llama a Submit() es el handler HTTP de POST /jobs. Si el inbox tuviera un límite y estuviera lleno, Submit() tendría que bloquearse hasta que hubiera lugar

y ese bloqueo lo sentiría, en vivo, el cliente que hizo el curl o el fetch(), como si el servidor se hubiera colgado. El inbox ilimitado es lo que garantiza que aceptar una tarea nueva y procesarla son dos cosas completamente desacopladas en el tiempo.

-El channel (task chan *Task):

Es la antesala inmediata a los workers, y **sí tiene límite** (channel-capacity, el flag que le pasás a main.go). Aquí es donde vive el backpressure real: si el channel ya está lleno, nadie puede meterle una tarea más hasta que algún worker saque una para procesarla.

-El dispatcher: El puente entre el Inbox y el Channel

Ninguna tarea salta directo del inbox al channel, hay una sola goroutine (dispatch) que hace de intermediaria; Saca una tarea del inbox con DequeueWait (Bloqueante, sin gastar CPU si no hay nada) y trata de meterla en el channel. Si el channel está lleno, dispatch simplemente espera ahí, en ese único punto. El inbox mientras tanto sigue aceptando tareas nuevas sin problema, porque Submit() nunca depende de que el channel tenga lugar.

4. cmd

4.1 cmd/server/main.go

El entrypoint. No tiene funciones propias más allá de main(), es pura composición; Arma todas las piezas de **internal/queue** e **internal/api** y las conecta.

Main() define las flags:

```
addr := flag.String("addr", ":8080", "direccion donde escucha el servidor")
numWorkers := flag.Int("workers", 4, "cantidad de workers concurrentes")
channelCapacity := flag.Int("channel-capacity", 16, "capacidad del channel bounded entre el dispatcher y los workers")
flag.Parse()
```

Arma el `context.Context` de shutdown con `signal.NotifyContext`, construye `DeadLetterQueue` → `WorkerPool` → `JobRegistry` (registrando "email" e "image_resize"), arranca el pool con `wp.Start(ctx)`, construye el Server de la API, levanta el `http.Server` en una goroutine aparte, y bloquea en `← ctx.Done()` esperando la señal de apagado para hacer `httpServer.Shutdown(shutdownCtx)` con un timeout de 5s.

Es el único lugar del proyecto que conoce `EmailJob` e `ImageResizeJob` por su nombre concreto — todo lo demás (el motor de la cola, los handlers HTTP) solo conoce la interfaz `Job`.

5. internal/queue

5.1 internal/queue/job.go

Define el contrato mínimo que cualquier trabajo debe cumplir, y el primer tipo concreto (simulado).

Job (interfaz) Un solo método: `Execute(ctx context.Context) error`. Es el Strategy del patrón, el motor de la cola llama `Execute()` sin saber ni importarle qué hay del otro lado.

5.2 internal/queue/status.go

Status (String) Los seis estados posibles de una tarea: **pending, processing, completed, failed, retrying, dead**. Es un tipo con nombre para que el compilador distinga "un estado de tarea" de "cualquier string", aunque por dentro sean idénticos.

5.3 internal/queue/queue.go

El corazón del dominio: la cola FIFO, el struct `Task`, y las funciones de generación de ID y backoff.

-NewQueue() *Queue Crea una cola vacía con su slice de items inicializado y el channel notify con buffer 1 (para el mecanismo de "avisar sin bloquear" que usa `Enqueue/DequeueWait`).

-NewTask(job Job) *Task Crea una `Task` en estado Pending, sin intentos, envolviendo el `Job` recibido, con un ID generado por `generateID()`.

-generateID() string Genera 8 bytes aleatorios con crypto/rand y los codifica en hexadecimal (16 caracteres). Si rand.Read fallara hace panic. Un fallo ahí indica algo grave del sistema operativo, no un caso a manejar con un error normal.

-(q *Queue) Len() int Devuelve la cantidad de items en la cola, tomando el lock.

-(q *Queue) Enqueue(task *Task) Agrega la tarea al final del slice (bajo lock), y después manda una señal no bloqueante por notify (select con default) para despertar a quien esté esperando en DequeueWait. El select+default evita apilar señales: si ya hay una pendiente sin consumir, no hace falta una segunda, con una alcanza para que el que espera vuelva a mirar toda la cola.

-(q *Queue) Dequeue() (*Task, error) Versión no bloqueante: si hay algo, lo saca y lo devuelve; si la cola está vacía, devuelve ErrEmptyQueue de inmediato.

-(q *Queue) DequeueWait(ctx context.Context) (*Task, error) Versión bloqueante, sin polling: intenta Dequeue(), y si falla, se queda esperando en un select entre $\leftarrow$ q.notify (reintentar) y $\leftarrow$ ctx.Done() (abortar con ctx.Err()). El loop externo es intencional, cubre wakeups espurios, nunca asume que despertar signifique que hay trabajo.

```
func (q *Queue) DequeueWait(ctx context.Context) (*Task, error) {
    for {
        task, err := q.Dequeue()
        if err == nil {
            return task, nil
        }
        select {
        case <-q.notify:
            // vuelve arriba del for, reintentar Dequeue() — puede fallar // de nuevo si otra
            // goroutine se adelantó a tomar esa tarea.
        case <-ctx.Done():
            return nil, ctx.Err()
        }
    }
}
```

-BackoffDuration(attempt int) time.Duration Calcula el delay antes del siguiente reintento: arranca en backoffBase (1s) y lo duplica attempt veces, con un techo de MaxBackoffDelay (30s). Usa un loop en vez de math.Pow para evitar overflow con exponentes grandes.

-(t *Task) Snapshot() TaskSnapshot Devuelve una copia inmutable y segura del estado actual de la tarea (bajo lock), con `LastError` convertido a string porque error no serializa bien a JSON. Es la única forma segura de leer una Task desde afuera de su procesamiento.

-(t *Task) ResetForRetry() Vuelve la tarea a Pending, `Attempts = 0`, `LastError = nil` — Usado en el reintento manual desde la DLQ, para darle un ciclo completo de intentos frescos en vez de uno solo.

5.4 internal/queue/worker.go

El motor de concurrencia: el pool de workers y el dispatcher.

-NewWorkerPool(numWorkers, channelCapacity int, dlq *DeadLetterQueue) *WorkerPool
Construye el pool con su inbox (`NewQueue()`), su channel bounded (`make(chan *Task, channelCapacity)`), la referencia a la DLQ recibida por fuera, y un TaskRegistry nuevo. No arranca ninguna goroutine, eso es responsabilidad de Start.

-(wp *WorkerPool) Submit(task *Task) La única puerta de entrada. Indexa la tarea en el registry y la encola en el inbox. Nunca bloquea al llamador, sin importar cuántas tareas entren de golpe, el inbox es ilimitado.

-(wp *WorkerPool) Pending() int Delega a `inbox.Len()` — cuántas tareas esperan todavía a ser tomadas por el dispatcher.

-(wp *WorkerPool) Start(ctx context.Context) Lanza dispatch como una goroutine, y `numWorkers` goroutines de `runWorker`. No bloquea al llamador.

-(wp *WorkerPool) dispatch(ctx context.Context) El loop que mueve tareas del inbox (ilimitado) al channel (bounded): `DequeueWait` para no hacer polling, y un select al escribir en `wp.tasks` para no quedar bloqueado para siempre si `ctx` se cancela justo cuando el channel está lleno.

-(wp *WorkerPool) runWorker(ctx context.Context) El loop de un worker individual: espera tareas del channel (`<-wp.tasks`, que bloquea sin polling), llama `ProcessTask`, y si la tarea queda en `Retrying`, delega el resto del ciclo a `scheduleRetry`.

```
func (wp *WorkerPool) runWorker(ctx context.Context) {  
    for {  
        select {  
            case <-ctx.Done():
```

```

        return
    case task := <-wp.tasks:
        ProcessTask(ctx, task, wp.dlq)
        if task.Status == StatusRetrying {
            delay := BackoffDuration(task.Attempts)
            wp.scheduleRetry(ctx, task, delay)
        }
    }
}

```

-(wp *WorkerPool) scheduleRetry(ctx context.Context, task *Task, delay time.Duration)

Lanza su propia goroutine que espera delay (con time.After) y vuelve a llamar Submit(task), el mismo camino que una tarea nueva. Corre aparte para no dejar inutilizado a un worker durante todo el backoff (hasta 30s).

-(wp *WorkerPool) Lookup(id string) (*Task, error) Delega a registry.Get(id), es lo que usa GET /jobs/:id para encontrar una tarea sin importar en qué estado esté.

5.5 internal/queue/process.go

-ProcessTask(ctx context.Context, task *Task, dlq *DeadLetterQueue) Ejecuta un único intento y decide el próximo estado. Pone Status = Processing, corre task.Job.Execute(ctx) **sin** sostener el lock durante la ejecución (Puede tardar lo que sea, y bloquear el lock ahí impediría leer Snapshot() mientras la tarea corre). Si no hubo error, pasa a Completed. Si hubo error, incrementa Attempts, guarda LastError, y si llegó a MaxAttempts pasa a Dead y la manda a la DLQ; si no, queda en Retrying para que runWorker programe el próximo intento.

```

func ProcessTask(ctx context.Context, task *Task, dlq *DeadLetterQueue) {
    task.mu.Lock()
    task.Status = StatusProcessing
    task.mu.Unlock()
    err := task.Job.Execute(ctx)
    if err == nil {
        task.mu.Lock()
        task.Status = StatusCompleted
        task.mu.Unlock()
        return
    }
}

```

```

    task.mu.Lock()
    task.Attempts++
    task.LastError = err
    attempts := task.Attempts
    if attempts >= MaxAttempts {
        task.Status = StatusDead
        task.mu.Unlock()
        dlq.Add(task)
        return
    }
    task.Status = StatusRetrying
    task.mu.Unlock()
}

```

5.6 internal/queue/registry.go

-NewTaskRegistry() *TaskRegistry Crea un registro vacío.

-(r *TaskRegistry) Add(task *Task) Indexa la tarea por ID. Se llama una sola vez, en Submit nunca se borra, ni siquiera en estados terminales.}

-(r *TaskRegistry) Get(id string) (*Task, error) Busca por ID; ErrTaskNotFound si no existe (mismo sentinel error que usa la DLQ).

5.7 internal/queue/dlq.go

Almacena las tareas que agotaron sus reintentos. A diferencia de Queue (FIFO estricta), permite listar todo su contenido y remover una tarea específica por ID, porque el caso de uso acá es "consultar y reintentar manualmente", no "procesar en orden".

-NewDeadLetterQueue() *DeadLetterQueue Crea una DLQ vacía con su map inicializado.

-(d *DeadLetterQueue) Add(task *Task) Guarda la tarea en el map, indexada por ID (bajo lock). Es lo que llama ProcessTask cuando una tarea llega a MaxAttempts.

-(d *DeadLetterQueue) List() []*Task Devuelve todas las tareas actuales como slice (bajo lock). El orden no está garantizado. Iterar un map en Go no respeta ningún orden fijo, así que dos llamadas seguidas a List() pueden devolver las mismas tareas en distinto orden.

-(d *DeadLetterQueue) Remove(id string) (*Task, error) Saca y devuelve la tarea con ese ID; si no existe, devuelve ErrTaskNotFound. Es el primer paso del reintento manual: sacarla de la DLQ antes de resetearla y volver a mandarla al pool.

6. internal/api

6.1 internal/api/server.go

-NewServer(wp *queue.WorkerPool, jobRegistry *queue.JobRegistry, dlq *queue.DeadLetterQueue) *Server Agrupa las tres dependencias que necesitan los handlers.

-(s *Server) Routes() *http.ServeMux Arma el mux con los 5 endpoints (POST /jobs, GET /jobs/{id}, GET /dlq, POST /dlq/{id}/retry, GET /dashboard), usando el http.ServeMux nativo de Go 1.22 con path variables.

6.2 internal/api/jobs.go

-(s *Server) handleCreateJob(w, r) Decodifica el body ({ "type", "payload" }), le pide al jobRegistry que construya el Job concreto, arma una Task nueva con queue.NewTask, la manda con wp.Submit, y responde 201 con el ID. 400 si el JSON es inválido o el tipo es desconocido.

-(s *Server) handleGetJob(w, r) Busca la tarea con wp.Lookup(id) y responde su Snapshot() (nunca los campos crudos, por la misma razón de siempre: un worker puede estar escribiéndolos en simultáneo). 404 si no existe.

6.3 internal/api/response.go

-writeJSON(w, status int, body any) Helper: setea Content-Type: application/json, escribe el status code, y codifica body como JSON en la respuesta. Lo usan todos los handlers para no repetir esas tres líneas.

-writeError(w, status int, msg string) Helper sobre writeJSON: envuelve msg en {"error": "..."}.